

Estructura de la Presentación Final

RUBÉN MARTÍNEZ POLLÁN
PABLO DíEZ FERNÁNDEZ
ADRIÁN GARCÍA DA SILVA

1. Introducción del Proyecto (3 min)

Nombre del producto: AIYama.

Problema: Los calendarios tradicionales son estáticos y rígidos. Estudiantes y profesionales tienen tareas flexibles (ej. "estudiar 2 horas") y es difícil encajarlas esquivando clases, horas de sueño y respetando los picos de energía de cada persona.

Público objetivo: Estudiantes universitarios, autónomos (freelancers) y cualquier persona con horarios mixtos que quiera maximizar su productividad sin quemarse.

Valor diferenciador: Es un Motor Híbrido. No le damos el calendario vacío a una IA para que alucine. Usamos matemáticas puras para calcular los huecos libres reales y luego usamos IA (Gemini) para aplicar psicología (cronotipos, energía) y elegir el mejor momento.

2. Formación del Equipo y Roles (1 min)

- **Integrantes:** Nuestro equipo está formado por **Rubén, Pablo y Adrián**. Hemos diseñado una estructura de trabajo colaborativa donde cada integrante cubre una capa específica del sistema, garantizando la interoperabilidad total entre el *frontend*, el *backend* y nuestra IA.
- **Roles y Responsabilidades:**
 - **Rubén** se ha centrado en el desarrollo de la API en Go, el diseño del motor algorítmico que calcula los huecos de tiempo y la integración con la IA de Gemini para procesar el lenguaje natural.
 - **Pablo** se ha encargado de la maquetación y desarrollo en React, creando la interfaz del calendario y el chatbot, además de programar servicios en el *backend* para la comunicación con la API.
 - **Adrián** ha gestionado el modelo de datos en PostgreSQL mediante Supabase, incluyendo las migraciones de esquemas, la lógica de autenticación completa y los elementos de navegación para la configuración de usuario.
- **Estrategia de colaboración:** se basa en tres frentes paralelos: Capa Visual, Capa de Datos y Capa Lógica. Trabajamos de forma asíncrona sobre módulos independientes y unificamos todo a través de GitHub mediante *Pull Requests* y

revisiones cruzadas, lo que nos permite asegurar la calidad y coherencia técnica en todas las capas del proyecto.

3. Requisitos y Backlog Inicial (2 min)

Requisitos Funcionales: Registro de usuarios, *Onboarding* (perfil de sueño y clases fijas), chat con IA, y renderizado del calendario semanal.

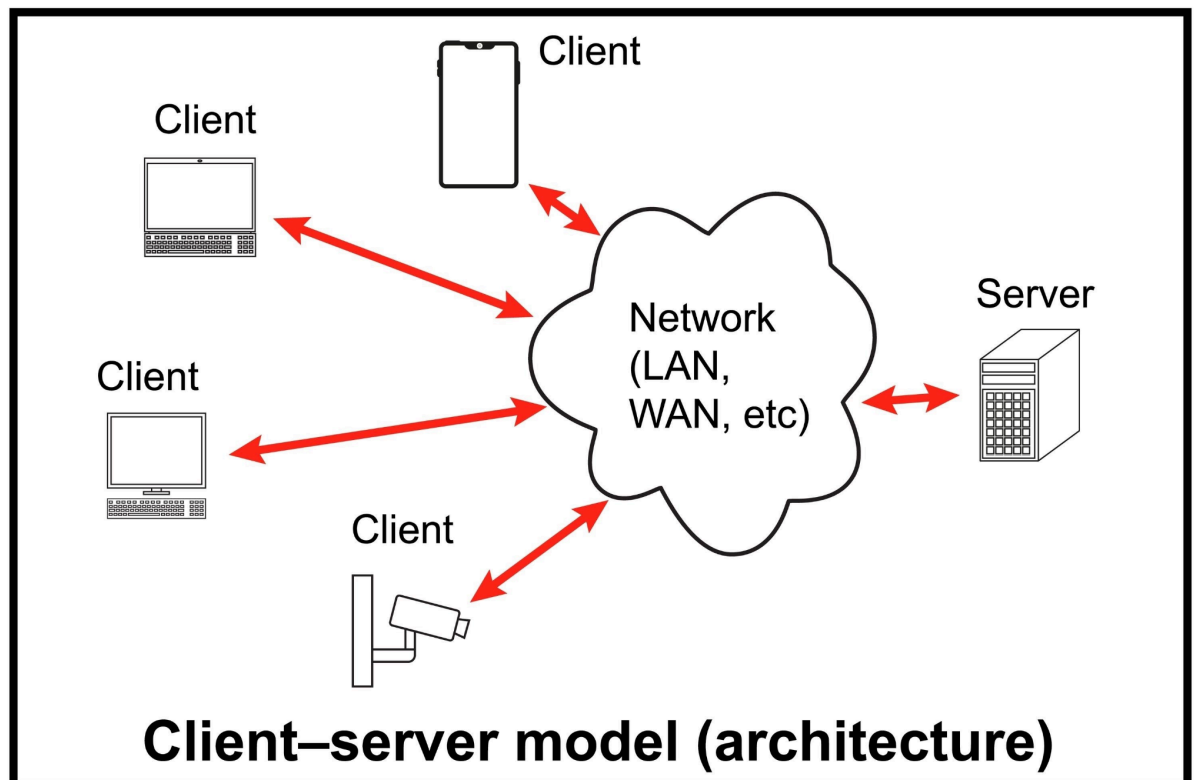
Requisitos No Funcionales: Persistencia de datos segura (JWT), tiempos de respuesta rápidos de la IA, y arquitectura escalable.

Organización en GitHub: Usamos GitHub Projects tipo Kanban (To Do, In Progress, Done).

Hito clave: El *Issue #15 (feat: create chatbot and consensus panel)*, donde conectamos finalmente el motor de Go, la API de Gemini y el estado interactivo de React.

4. Diseño y Arquitectura (3 min)

- Diagrama:



Arquitectura: Cliente-Servidor (API REST).

Tecnologías:

- **Frontend:** React con TypeScript (Vite). Permite crear componentes reutilizables (como el calendario) y tipado estricto para evitar errores.
- **Backend:** Go (con el framework Gin). Go es extremadamente rápido y eficiente, ideal para el motor matemático que calcula los huecos de tiempo.
- **Base de Datos:** PostgreSQL alojada en Supabase.
- **IA:** Google Gemini API.

El sistema gestiona la información a través de **tres entidades fundamentales** que permiten la planificación inteligente:

- **User:** Almacena el perfil del usuario, incluyendo el cronotipo (mañana/tarde/noche), el objetivo de horas de sueño y los horarios preferentes de descanso. Esta información es la base sobre la que el motor algorítmico toma decisiones.
- **FixedBlock:** Representa las rutinas inamovibles del usuario (como jornadas laborales o clases). Estos bloques actúan como restricciones "hard" que el motor algorítmico tiene estrictamente prohibido solapar al asignar tareas.
- **FlexibleTask:** Representa las actividades flexibles que el usuario desea realizar. A diferencia de un bloque fijo, esta entidad almacena el estado de planificación (si ha sido asignada o no) y permite al motor de la IA calcular el hueco óptimo basándose en la disponibilidad y la energía del usuario.

5. Implementación y Buenas Prácticas (4 min)

- **Flujo de trabajo y colaboración:** Para el desarrollo del proyecto, el equipo adoptó una metodología de trabajo colaborativo basada en el control de versiones con Git, lo cual fue vital para gestionar el desarrollo concurrente. Mientras Adrián se encargaba del modelado de datos en Supabase, garantizando la escalabilidad del esquema, Pablo se centraba en la maquetación del frontend en React y ayudaba a Rubén mientras construía la lógica del backend en Go. Esta división de responsabilidades, combinada con el uso de ramas independientes (*feature branches*), permitió un desarrollo paralelo sin conflictos.
- **Control de calidad y revisión:** La regla de oro del equipo fue la implementación de *Pull Requests* (PRs) como mecanismo obligatorio de control de calidad. Ninguna pieza de código era integrada en la rama principal (*main*) sin haber pasado previamente por una revisión cruzada entre los otros dos miembros del equipo. Este proceso no solo minimizó la aparición de errores en producción, sino que facilitó el aprendizaje continuo y la transferencia de conocimiento entre todos nosotros.
- **Estandarización del entorno:** Para asegurar la consistencia, todo el entorno está completamente dockerizado. El uso de *Dockerfile* y *docker-compose.yml* permitió estandarizar el despliegue tanto en entornos locales de desarrollo como en la infraestructura en la nube. Esta práctica eliminó las discrepancias de configuración, garantizando que el sistema fuera robusto y portable desde el primer día.

6. CI/CD y Despliegue (3 min)

- **Pipeline (Integración Continua):** Implementamos GitHub Actions (*ci.yml*). Cada vez que hacíamos un *push* o abríamos una *Pull Request*, se disparaba un flujo automático que compilaba el código en Go y ejecutaba los tests. Esto actuaba como un escudo protector para asegurar que ningún cambio nuevo rompiera la aplicación.
- **Contenedores (Docker):** Todo nuestro entorno está completamente dockerizado (usando *Dockerfile* y *docker-compose.yml*). Esto nos permitió eliminar el clásico problema de "en mi máquina sí funciona", estandarizando el paso del entorno de desarrollo a producción.
- **Despliegue en Render:** Aprovechando esta arquitectura de contenedores, elegimos **Render** para alojar la aplicación web. Al tener el backend dockerizado, el proceso fue súper fluido: Render se encarga de coger nuestra imagen de Docker, levantarla de forma automática con cada actualización y exponer tanto nuestro servidor en Go como el frontend al mundo real de manera estable.

7. Demostración del Producto (5 min)

- **Login/Onboarding:** Mostrar cómo un usuario nuevo entra y configura que es "Alondra" (rinde de mañana) y que duerme de 23:00 a 07:00.
- **Dashboard:** Mostrar el calendario vacío solo con las clases fijas.
- **Interacción Core:** Escribir en el chat: "Quiero x".
- **Demostración:** Mostrar cómo la IA responde con una propuesta coherente.
- **Consenso:** Darle al botón "Sí" y ver cómo aparece mágicamente dibujado en el calendario de Pablo.

8. Documentación y Experiencia de Desarrollo (2 min)

- **Documentación:** hemos entregado un documento de Especificación de Requisitos de Software (SRS) exhaustivo. Este archivo es la hoja de ruta del proyecto. En él, el cliente puede encontrar detallados todos los requisitos funcionales —como la gestión de tareas, el motor de asignación por IA y el sistema de *rollback*—, junto con la descripción de las tecnologías empleadas. Hemos definido desde las interfaces hasta los requisitos no funcionales, como la seguridad y el rendimiento, asegurando que el cliente tenga una visión clara, unívoca y profesional de lo que es Alyama.
- **Desarrollo: Integración Continua:**

Todo nuestro código pasa por *Pull Requests* en GitHub. Nadie sube nada sin que el resto del equipo revise y valide los cambios, lo que nos ha permitido mantener una coherencia técnica total y minimizar errores.

Comunicación Síncrona: A pesar de haber dividido el trabajo en capas —visual, lógica y de datos—, hemos mantenido reuniones frecuentes para alinear las piezas del *backend* en Go con la interfaz de React.

Gestión del Proyecto: Esta estructura nos ha permitido trabajar de forma ágil y asíncrona, pero manteniendo a todo el equipo al tanto de cualquier bloqueo o avance. Lo más importante fue cómo logramos coordinarnos. Pasamos de trabajar cada uno en nuestro bloque a integrar el diseño de datos, la lógica del backend y la interfaz de usuario en un solo engranaje. Fue clave revisar el código entre todos para que, cuando el diseño de Adrián se conectaba con la IA de Rubén y lo que hacía Pablo en el front, todo fluyera sin errores.

9. Conclusión y Preguntas (1 min)

- **Estado final:** Un MVP (Producto Mínimo Viable) totalmente funcional que demuestra que la IA puede integrarse de forma segura en herramientas de productividad.
- **Mejoras futuras:** Integración real con Google Calendar, tareas recurrentes automáticas y soporte para arrastrar eventos (*drag and drop*) en el frontend para reajustar lo que dijo la IA.